

LLM labeling of STM topics and design-based stance measurement

```
suppressPackageStartupMessages({
  library(here); library(survey); library(splines); library(viridis)
  library(tidyverse)      # LAST so its verbs win
})
# Namespaced only: ellmer (LLM calls), furr/future (the ONE parallel step).
theme_set(theme_minimal(base_size = 12))
# Tables must not silently lose rows in the rendered PDF.
options(knitr.table.max.print = Inf, knitr.kable.max_rows = Inf,
        max.print = 1e6, survey.lonely.psu = "adjust")

st <- list(
  seed          = 2026L,
  proj          = "text_mining",      # project folder under the here() root;
                                      # the one string to change when reusing
  simulated     = TRUE,               # THE one-flag swap

  # --- models: label/propositions, stance rater 1, stance rater 2 -----
  model_label   = "openrouter/meta-llama/llama-4-maverick",
  model_stance  = c(gemma4 = "openrouter/google/gemma-4-31b-it",
                    llama31 = "openrouter/meta-llama/llama-3.1-70b-instruct"),
  persona       = str_c("a survey methodologist who codes open-ended ",
                        "responses for a public research program"),
  show_year     = TRUE,               # year shown for TOPIC tasks (context helps naming)
  show_theta    = FALSE,              # theta shown to the STANCE rater. Default off: the
                                      # relevance gate already handles off-target text,
                                      # and a topic-membership number invites the model to
                                      # reason about relevance instead of position.

  # --- scope and sampling -----
  min_theta     = 0.30,               # modal-assignment floor (diffuse paragraphs out)
  stance_topics = NULL,               # e.g. c(3L, 7L, 11L). NULL: top 3 by prevalence
  e_target      = 0.075,              # target half-width for a topic-level proportion
  icc           = 0.30,               # within-comment correlation assumption (deff input)
  b_paras       = 3L,                 # paragraphs per sampled comment
  floor_comments = 2L,                # minimum comments sampled per year stratum
  n_gold        = 100L,               # paragraphs per topic sent for human coding
  calls_window  = 300L,               # API quota per 4-hour window PER model
  retries       = 1L,                 # PRODUCTION: 16L
  retry_wait    = 5L                  # PRODUCTION: 30L * 60L
)

set.seed(st$seed)
```

```

# All paths anchored with here(): <here()>/text_mining/{code, input, outputs}.
# Both scripts write to the SAME outputs folder, so Script 2 reads Script 1's
# files through the same helper.
p_out <- function(f) here::here(st$proj, "outputs", f)
p_in  <- p_out
dir.create(here::here(st$proj, "outputs"), showWarnings = FALSE,
           recursive = TRUE)
cache_csv <- function(path, expr) {
  if (file.exists(path)) return(readr::read_csv(path, show_col_types = FALSE))
  x <- force(expr); readr::write_csv(x, path); x
}
source(here::here(st$proj, "code", "llm_source.R"))

paragraph_theta <- readr::read_csv(p_in("paragraph_theta.csv"),
                                   show_col_types = FALSE)
topic_summary    <- readr::read_csv(p_in("topic_summary.csv"),
                                   show_col_types = FALSE)
stopifnot("Script 1 outputs must carry modal topic and theta_max" =
  all(c("topic", "theta_max", "para_uid", "atom_id", "Year",
        "text") %in% names(paragraph_theta)))

```

Design of the measurement

STM is the topic method: it runs on the whole corpus, costs no API calls, is reproducible, and assigns every paragraph a modal topic. The LLM does three jobs it is better at than a bag-of-words model, and no others: naming what STM found, drafting propositions a human then edits, and reading stance from text. Labels never enter an estimate, so a mislabeled topic is a communication error, not a measurement error.

Every prompt has the same four parts. **ROLE** frames the expertise so the model reads as the right kind of analyst. **TASK** states the job and a self-check step. **RULES** is the anti-hallucination slot: only the shown evidence, no invented specifics, no drift into adjacent questions. **OUTPUT** states the return shape and is paired with a structured schema, so an unparseable answer is impossible rather than merely discouraged.

Topic labels

One call per topic. The model sees that topic's FREX and highest-probability terms and its top paragraphs, and returns a short label and one factual description. It is blind to Year, to stance, and to every other topic's words, so labels cannot be harmonized into a narrative the data does not support. The result is written to `topic_codebook.csv` for you to edit; that file, not this model, is the naming record.

```

top_paras <- paragraph_theta |>
  dplyr::group_by(topic) |>
  dplyr::slice_max(theta_max, n = 3) |>

```

```

dplyr::summarise(ex = str_c(str_trunc(text, 300), collapse = "\n- "),
                 .groups = "drop")

label_sys <- build_prompt(
  role = str_c("You are ", st$persona,
               ". You name statistical topics so colleagues can use them."),
  task = str_c("Read the topic's distinctive terms and example paragraphs. ",
               "Write a short label and one factual sentence describing what ",
               "these paragraphs are about. Before answering, check that ",
               "every word of your description is supported by the evidence ",
               "shown."),
  rules = c("Use only the terms and paragraphs shown.",
            "Do not infer stance, sentiment, intent, or who is speaking.",
            "Do not invent statistics, dates, places, or named entities.",
            "If the evidence is mixed, describe the mixture plainly."),
  output = "A label of at most 4 words and a one-sentence description.")

label_schema <- ellmer::type_object(
  label = ellmer::type_string("Topic name, at most 4 words."),
  description = ellmer::type_string("One factual sentence.))

codebook <- cache_csv(p_out("topic_codebook.csv"), {
  ev <- topic_summary |> dplyr::left_join(top_paras, by = "topic")
  prompts <- str_c("Distinctive (FREX) terms: ", ev$frex,
                  "\nHighest-probability terms: ", ev$prob,
                  "\nExample paragraphs:\n- ", ev$ex)
  llm_task(prompts, ev$topic, label_schema, st$model_label, label_sys,
            st$retries, st$retry_wait, st$simulated,
            fake = function(ids) tibble(
              id = ids, ok = TRUE,
              label = str_c("Topic ", ids, " theme"),
              description = str_c("Paragraphs about theme ", ids, ".))) |>
  dplyr::transmute(topic = id, label, description) |>
  dplyr::left_join(topic_summary |>
    dplyr::select(topic, prevalence, n_paras, avepp, frex),
    by = "topic")
})
codebook |>
  dplyr::mutate(dplyr::across(c(prevalence, avepp), ~ round(.x, 3))) |>
  dplyr::arrange(dplyr::desc(prevalence)) |>
  dplyr::select(topic, label, description, prevalence, n_paras, avepp) |>
  knitr::kable(caption = "Topic codebook: LLM drafts plus the STM evidence a
human needs to check them. Edit topic_codebook.csv; it is the naming record for
coders and write-ups.")

```

topic	label		description	prevalence	n_ paras	avepp
24	Topic theme	24	Paragraphs about theme 24.	0.046	381	0.963
13	Topic theme	13	Paragraphs about theme 13.	0.045	370	0.962
22	Topic theme	22	Paragraphs about theme 22.	0.045	374	0.962
23	Topic theme	23	Paragraphs about theme 23.	0.045	367	0.962
25	Topic theme	25	Paragraphs about theme 25.	0.044	375	0.897
15	Topic theme	15	Paragraphs about theme 15.	0.043	352	0.962
16	Topic theme	16	Paragraphs about theme 16.	0.043	351	0.962
20	Topic theme	20	Paragraphs about theme 20.	0.043	355	0.962
2	Topic 2 theme		Paragraphs about theme 2.	0.041	335	0.961
5	Topic 5 theme		Paragraphs about theme 5.	0.041	339	0.961
7	Topic 7 theme		Paragraphs about theme 7.	0.041	338	0.961
21	Topic theme	21	Paragraphs about theme 21.	0.041	335	0.961
11	Topic theme	11	Paragraphs about theme 11.	0.040	326	0.961
3	Topic 3 theme		Paragraphs about theme 3.	0.039	321	0.961
6	Topic 6 theme		Paragraphs about theme 6.	0.039	319	0.960
10	Topic theme	10	Paragraphs about theme 10.	0.038	312	0.960
18	Topic theme	18	Paragraphs about theme 18.	0.038	315	0.960
4	Topic 4 theme		Paragraphs about theme 4.	0.037	301	0.960
12	Topic theme	12	Paragraphs about theme 12.	0.037	300	0.960

topic	label		description	prevalence	n_ paras	avepp
14	Topic theme	14	Paragraphs about theme 14.	0.037	303	0.960
9	Topic 9 theme		Paragraphs about theme 9.	0.036	299	0.960
17	Topic theme	17	Paragraphs about theme 17.	0.036	296	0.960
19	Topic theme	19	Paragraphs about theme 19.	0.036	297	0.960
1	Topic 1 theme		Paragraphs about theme 1.	0.035	289	0.960
8	Topic 8 theme		Paragraphs about theme 8.	0.035	286	0.960

Table 1: Topic codebook: LLM drafts plus the STM evidence a human needs to check them. Edit `topic_codebook.csv`; it is the naming record for coders and write-ups.

Propositions for stance

Stance is a position toward a stated claim, so each selected topic needs one. The model drafts three candidates per topic; you keep or rewrite one. The binding rule, enforced in the prompt and checked in code: a proposition must be a **contestable directional claim** that a paragraph can favor or oppose. A topic restatement (“this is about funding”) drives every paragraph to neutral and silently destroys the measurement.

```
sel <- st$stance_topics %||% (codebook |> dplyr::slice_max(prevalence, n = 3) |
>
                                dplyr::pull(topic))

prop_sys <- build_prompt(
  role = str_c("You are ", st$persona, ", drafting stance items."),
  task = str_c("From the topic label, description, and terms, draft three ",
    "candidate propositions. Each must be a claim a commenter ",
    "could plausibly support or oppose. Before answering, test ",
    "each candidate: if a paragraph could not disagree with it, ",
    "rewrite it."),
  rules = c("A proposition states a direction, not a subject.",
    "Use only the evidence shown; invent no policies or programs.",
    "Keep each under 20 words and free of loaded language.",
    "Do not restate the topic label as a proposition."),
  output = "Three candidate propositions.")

prop_schema <- ellmer::type_object(
  p1 = ellmer::type_string("Candidate 1."),
  p2 = ellmer::type_string("Candidate 2."),
  p3 = ellmer::type_string("Candidate 3."))
```

```

propositions <- cache_csv(p_out("topic_propositions.csv"), {
  ev <- codebook |> dplyr::filter(topic %in% sel)
  prompts <- str_c("Label: ", ev$label, "\nDescription: ", ev$description,
    "\nDistinctive terms: ", ev$frex)
  llm_task(prompts, ev$topic, prop_schema, st$model_label, prop_sys,
    st$retries, st$retry_wait, st$simulated,
    fake = function(ids) tibble(
      id = ids, ok = TRUE,
      p1 = str_c("Programs related to theme ", ids,
        " should be expanded."),
      p2 = str_c("Funding for theme ", ids, " should increase."),
      p3 = str_c("Theme ", ids, " policy is working well.))) |>
    dplyr::transmute(topic = id, proposition = p1, alt_1 = p2, alt_2 = p3) |>
    dplyr::left_join(codebook |> dplyr::select(topic, label), by = "topic") |>
    dplyr::relocate(topic, label, proposition)
})
# Cheap contestability check: a directional claim almost always carries a
# modal or comparative verb. Flags, does not block; the analyst decides.
propositions |>
  dplyr::mutate(flag_not_contestable = !str_detect(
    str_to_lower(proposition),
    "should|must|ought|more|less|increase|decrease|expand|reduce|better|
worse")) |>
  dplyr::select(topic, label, proposition, flag_not_contestable) |>
  knitr::kable(caption = "Drafted propositions for the selected topics. Edit
topic_propositions.csv: the proposition column is the measurement target and is
the analyst's, not the model's. Flagged rows lack a directional cue.")

```

topic	label	proposition	flag_not_ contestable
13	Topic theme	13 Programs related to theme 13 should be expanded.	FALSE
22	Topic theme	22 Programs related to theme 22 should be expanded.	FALSE
24	Topic theme	24 Programs related to theme 24 should be expanded.	FALSE

Table 2: Drafted propositions for the selected topics. Edit `topic_propositions.csv`: the proposition column is the measurement target and is the analyst's, not the model's. Flagged rows lack a directional cue.

Sampling design

Stance is measured on a probability sample, not the corpus, because the API budget is 300 calls per 4-hour window per model and every estimate must generalize to the corpus anyway. The

design is a **stratified two-stage cluster sample within each selected topic**: the frame is that topic's paragraphs with modal theta at least 0.3, strata are **years**, the primary sampling unit is the **comment**, and 3 paragraphs per sampled comment are drawn by SRS. Inclusion probabilities are $\pi_{hij} = (n_h/N_h) \times (b/m_i)$ and weights are $w = 1/\pi$, so estimates describe the topic's corpus, not the sample.

Allocation is proportionate to each year's comment count with a floor of 2 comments, which is how the design answers the uneven-years problem: a year with few comments is still sampled, and the weights undo the disproportionality rather than the estimate quietly dropping the year. Sample size is not chosen by feel. For a proportion,

$$n_0 = \text{deff} \cdot \frac{z^2 p(1-p)}{e^2}, \quad n = \frac{n_0}{1 + n_0/N}, \quad \text{deff} = 1 + (b-1)\rho,$$

with $z = 1.96$, $p = 0.5$ (maximum variance), $e = 0.075$, $\rho = 0.3$, and N the topic's frame size. The half-width applies to a topic-level proportion; on the fitted year curve, intervals at a given year are wider, which is why trends rather than single years are the reportable claim.

```
frame <- paragraph_theta |>
  dplyr::filter(topic %in% sel, theta_max >= st$min_theta) |>
  dplyr::select(para_uid, atom_id, Year, n_tokens, text, topic, theta_max)

sample_paras <- cache_csv(p_out("sample_paragraphs.csv"), {
  frame |>
    dplyr::group_split(topic) |>
    purrr::map(function(pool) {
      n_t <- n_for_precision(nrow(pool), st$e_target, st$icc, st$b_paras)
      draw_topic_sample(pool, n_t, st$b_paras, st$floor_comments,
        st$seed + dplyr::first(pool$topic))
    }) |>
    purrr::list_rbind()
})

sample_paras |>
  dplyr::group_by(topic) |>
  dplyr::summarise(frame_paras = NA_integer_, sampled = dplyr::n(),
    comments = dplyr::n_distinct(atom_id),
    years = dplyr::n_distinct(Year),
    min_w = min(w), max_w = max(w), .groups = "drop") |>
  dplyr::select(-frame_paras) |>
  dplyr::left_join(frame |> dplyr::count(topic, name = "frame_paras"),
    by = "topic") |>
  dplyr::left_join(codebook |> dplyr::select(topic, label), by = "topic") |>
  dplyr::relocate(topic, label, frame_paras, sampled, comments, years) |>
  dplyr::mutate(dplyr::across(c(min_w, max_w), ~ round(.x, 2))) |>
  knitr::kable(caption = "Realized sample by topic: frame size, sampled
  paragraphs, comment clusters, years covered, and the weight range. Unequal
```

weights are the price of the allocation floor and are carried into every estimate.")

topic	label		frame_ paras	sam- pled	com- ments	years	min_w	max_w
13	Topic theme	13	370	160	132	26	1.5	3.05
22	Topic theme	22	374	155	130	26	2.0	2.67
24	Topic theme	24	381	162	135	26	1.0	3.11

Table 3: Realized sample by topic: frame size, sampled paragraphs, comment clusters, years covered, and the weight range. Unequal weights are the price of the allocation floor and are carried into every estimate.

```
glue::glue("Stance calls per model: {nrow(sample_paras)} ",
           "({ceiling(nrow(sample_paras) / st$calls_window)} quota windows). ",
           "The two raters spend their windows concurrently, so wall time is ",
           "one rater's, not the sum.")
```

Stance calls per model: 477 (2 quota windows). The two raters spend their windows concurrently, so wall time is one rater's, not the sum.

SIMULATED MODE (delete this chunk at work, or set `st$simulated = FALSE`)

Plants a stance truth per sampled paragraph, with a per-topic linear trend in Year, so the estimators can be scored against a known answer. The fake raters and the fake human coder read that truth through noise channels of different fidelity. This validates the pipeline and the estimators; it says nothing about how the real models perform on real text, and the prose keeps those claims separate.

```
if (st$simulated) {
  set.seed(st$seed)
  slope <- set_names(seq(-0.015, 0.015, length.out = length(sel)), sel)
  sample_paras <- sample_paras |>
    dplyr::mutate(
      p_opp = pmin(.85, pmax(.05,
                            .35 + slope[as.character(topic)] * (Year - 2000))),
      true_stance = purrr::map_chr(p_opp, ~ sample(
        c("favor", "neutral", "oppose", "irrelevant"), 1,
```



```

    prob = c(.55 * (1 - .x), .35 * (1 - .x), .x, .10 * (1 - .x))))
  }

```

Stance detection

One paragraph per call, never batched: items sharing a context bleed into each other, and stance is the measurement where that matters most. Two raters, run **concurrently** across models (the one parallel step in the pipeline, justified because each model has its own quota, so concurrency halves wall time without raising any per-model rate) and strictly sequential within each. The prompt shows the paragraph and its proposition. Year is withheld by default and `st$show_theta` is off: telling a rater the country-or-period, or how strongly a paragraph loads on a topic, invites it to infer position from context rather than read it from text, and the `irrelevant` label already handles off-target paragraphs.

```

stance_labels <- c("favor", "neutral", "oppose", "irrelevant")
stance_schema <- ellmer::type_object(
  stance = ellmer::type_enum(stance_labels, "The single best-fitting label.")

stance_sys <- function(prop) build_prompt(
  role = str_c("You are ", st$persona, ", coding stance in public comments."),
  task = str_c("Decide the author's position toward this proposition: \"",
    prop, "\". First check whether the text addresses the ",
    "proposition at all; if it does not, the label is irrelevant. ",
    "Then decide favor, oppose, or neutral."),
  rules = c("Stance is not tone: an angry passage can favor the proposition.",
    "Judge only the text shown; use no outside knowledge.",
    "Do not explain, quote, or add anything to the output.",
    "If the text is ambiguous, choose neutral rather than guessing."),
  output = "One label: favor, neutral, oppose, or irrelevant.")

stance_raw <- cache_csv(p_out("stance_labels.csv"), {
  future::plan(future::multisession, workers = length(st$model_stance))
  out <- furrr::future_imap(st$model_stance, function(mdl, nm) {
    sample_paras |>
      dplyr::group_split(topic) |>
      purrr::map(function(d) {
        prop <- propositions$proposition[propositions$topic ==
          dplyr::first(d$topic)]
        prompts <- str_c(if (st$show_year) str_c("Year: ", d$Year, "\n") else
          "",
          if (st$show_theta)
            str_c("Topic loading: ", round(d$theta_max, 2), "\n")
          else "",
          "Text: ", d$text)
        llm_task(prompts, d$para_uid, stance_schema, mdl, stance_sys(prop),
          st$retries, st$retry_wait, st$simulated,

```

```

fake = function(ids) tibble(
  id = ids, ok = TRUE,
  stance = purrr::map_chr(
    d$true_stance[match(ids, d$para_uid)],
    ~ if (runif(1) < .85) .x
      else sample(setdiff(stance_labels, .x), 1))) |>
  dplyr::transmute(para_uid = id, model = nm, stance)
}) |> purrr::list_rbind()
}, .options = furrr::furrr_options(seed = TRUE))
future::plan(future::sequential)
purrr::list_rbind(out)
})

wide <- stance_raw |>
  tidyr::pivot_wider(names_from = model, values_from = stance)
ag <- agreement(wide[[2]], wide[[3]], dnn = names(wide)[2:3])
glue::glue("Inter-rater agreement over {ag$n} paragraphs: kappa = ",
  "{round(ag$kappa, 3)}, raw agreement = {round(ag$raw, 3)}. ",
  "Kappa has no validated interpretive thresholds, so the confusion ",
  "table below carries the detail.")

```

Inter-rater agreement over 477 paragraphs: kappa = 0.607, raw agreement = 0.719.
Kappa has no validated interpretive thresholds, so the confusion table below carries the detail.

```
ag$confusion
```

	llama31			
gemma4	favor	irrelevant	neutral	oppose
favor	126	13	11	14
irrelevant	5	19	12	10
neutral	13	9	77	14
oppose	11	9	13	121

Human coding

Agreement between two models is reliability, not accuracy: both can be wrong together. The only thing that turns “the raters agreed” into “the raters were right” is human labels. `gold_to_code.csv` is written with the paragraph text, its topic label, and the proposition, and with `human_label` blank: the coder sees exactly what the model saw and no model output, so the comparison is not anchored. Save the completed file as `gold_coded.csv` and re-render.

```

gold_ids <- sample_paras |>
  dplyr::group_by(topic) |>

```

```

dplyr::mutate(.r = sample(dplyr::n())) |>
dplyr::filter(.r <= st$n_gold) |> dplyr::ungroup() |> dplyr::pull(para_uid)

gold_to_code <- sample_paras |>
dplyr::filter(para_uid %in% gold_ids) |>
dplyr::left_join(codebook |> dplyr::select(topic, label, description),
  by = "topic") |>
dplyr::left_join(propositions |> dplyr::select(topic, proposition),
  by = "topic") |>
dplyr::transmute(para_uid, topic, topic_label = label, topic_description =
  description, proposition, text,
  human_label = NA_character_, coder_notes = NA_character_)
readr::write_csv(gold_to_code, p_out("gold_to_code.csv"))

# Simulated mode fills a SEPARATE, clearly named file. A file with SIMULATED
# in its name and its source column can never be mistaken for human work.
if (st$simulated)
  gold_to_code |>
    dplyr::left_join(sample_paras |> dplyr::select(para_uid, true_stance),
      by = "para_uid") |>
    dplyr::mutate(human_label = purrr::map_chr(true_stance, ~ if (runif(1) < .93)
      .x else sample(setdiff(stance_labels, .x), 1)),
      source = "SIMULATED") |>
    dplyr::select(-true_stance) |>
    readr::write_csv(p_out("gold_coded_SIMULATED.csv"))

gold_file <- c(p_out("gold_coded.csv"), p_out("gold_coded_SIMULATED.csv")) |>
  purrr::keep(file.exists) |> head(1)
gold <- if (length(gold_file)) readr::read_csv(gold_file,
  show_col_types = FALSE) else NULL
if (!is.null(gold) && str_detect(gold_file, "SIMULATED"))
  warning("Validity metrics below come from SIMULATED codes, not a human ",
    "coder. They validate the machinery only.")

if (!is.null(gold)) {
  purrr::iwalk(st$model_stance, function(mdl, nm) {
    j <- stance_raw |> dplyr::filter(model == nm) |>
      dplyr::inner_join(gold |> dplyr::select(para_uid, human_label),
        by = "para_uid")
    m <- class_metrics(j$stance, j$human_label)
    k <- agreement(j$stance, j$human_label, dnn = c(nm, "human"))
    glue::glue("[{nm}] macro-F1 {round(m$macro_f1, 3)}, accuracy ",
      "{round(m$accuracy, 3)}, kappa vs human {round(k$kappa, 3)}") |>
      print()
    print(m$per_class |> dplyr::mutate(dplyr::across(
      c(precision, recall, f1), ~ round(.x, 3))))
  })
} else {

```

```

glue::glue("No coded file yet. Code gold_to_code.csv, save it as \\
           gold_coded.csv, and re-render. Until then the stance labels \\
           are unvalidated.")
}

```

```

[gemma4] macro-F1 0.766, accuracy 0.8, kappa vs human 0.718
# A tibble: 4 × 5
  class      precision recall    f1 support
<chr>      <dbl>    <dbl> <dbl>    <int>
1 favor      0.777    0.851 0.812     94
2 irrelevant 0.562    0.72  0.632     25
3 neutral    0.797    0.729 0.761     70
4 oppose     0.901    0.82  0.858    111
[llama31] macro-F1 0.76, accuracy 0.8, kappa vs human 0.719
# A tibble: 4 × 5
  class      precision recall    f1 support
<chr>      <dbl>    <dbl> <dbl>    <int>
1 favor      0.837    0.819 0.828     94
2 irrelevant 0.515    0.68  0.586     25
3 neutral    0.764    0.786 0.775     70
4 oppose     0.883    0.82  0.85     111

```

Stance over time

The estimand is a smooth trend, not 26 separate yearly proportions: with roughly 6 sampled paragraphs per topic-year, isolated annual estimates are too thin to interpret, while a spline lets every year's paragraphs inform the curve. `svyglm` with a quasibinomial link fits $P(\text{oppose})$ on a natural spline of Year under the full design (year strata, comment clusters, weights), so the fitted curve and its interval are design-based. The with-replacement approximation is used, which is conservative.

```

est <- sample_paras |>
  dplyr::inner_join(
    stance_raw |> dplyr::filter(model == names(st$model_stance)[1]) |>
      dplyr::select(para_uid, stance), by = "para_uid") |>
  dplyr::mutate(oppose = as.integer(stance == "oppose")) |>
  dplyr::left_join(codebook |> dplyr::select(topic, label), by = "topic")

fit_topic <- function(d) {
  des <- svydesign(ids = ~ atom_id, strata = ~ Year, weights = ~ w,
    data = d, nest = TRUE)
  m <- svyglm(oppose ~ ns(Year, df = 3), design = des,
    family = quasibinomial())
  nd <- tibble(Year = sort(unique(d$Year)))
  pr <- predict(m, newdata = nd, type = "link", se.fit = TRUE)
  # SE() returns a 1-column matrix; as.numeric() keeps the output columns

```

```

# atomic so the table writes to CSV cleanly.
lp <- as.numeric(pr); se <- as.numeric(SE(pr))
nd |> dplyr::mutate(label = dplyr::first(d$label),
                   fit = plogis(lp),
                   lo = plogis(lp - 1.96 * se),
                   hi = plogis(lp + 1.96 * se))
}
trends <- est |> dplyr::group_split(topic) |>
  purrr::map(fit_topic) |> purrr::list_rbind()
readr::write_csv(trends, p_out("stance_trends.csv"))

p <- ggplot(trends, aes(Year, fit, color = label, fill = label)) +
  geom_ribbon(aes(ymin = lo, ymax = hi), alpha = .15, color = NA) +
  geom_line(linewidth = 1) +
  scale_color_viridis_d(end = .85, name = NULL) +
  scale_fill_viridis_d(end = .85, guide = "none") +
  labs(x = NULL, y = "P(oppose)")
if (st$simulated)
  p <- p + geom_line(
    data = sample_paras |>
      dplyr::left_join(codebook |> dplyr::select(topic, label), by = "topic") |
    >
    dplyr::distinct(label, Year, p_opp),
    aes(Year, p_opp, color = label), linetype = 2, linewidth = .5,
    show.legend = FALSE)
print(p)

```

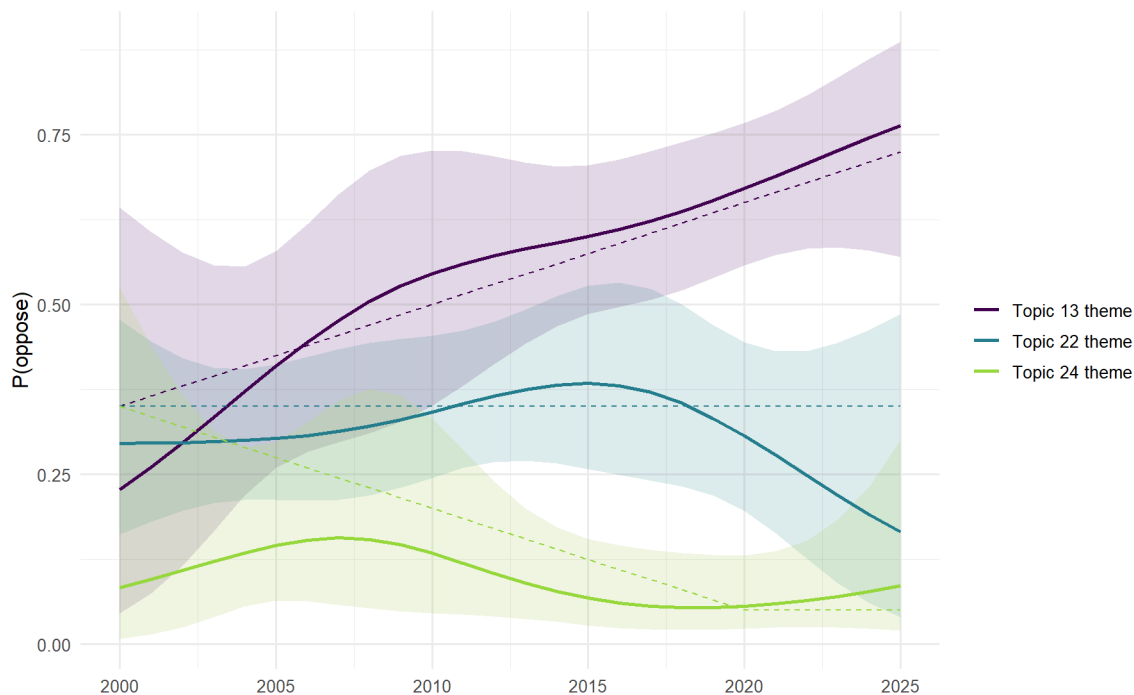


Figure 1: Design-based $P(\text{oppose})$ by year and topic with 95 percent intervals from svyglm. Dashed lines in simulated mode are the planted truth; the vertical gap is measurement attenuation from rater error, which the human-coded metrics above quantify.

Sensitivity to rater misclassification

Design intervals cover sampling error only. Rater error is a separate, larger source, and it attenuates proportions toward the majority class. For a binary outcome with symmetric error rate m , the observed proportion relates to the truth by $p_{obs} = p(1 - m) + (1 - p)m$, so $p = (p_{obs} - m)/(1 - 2m)$. The table shows the pooled estimate corrected at assumed error rates; if a conclusion survives all three, it is robust to plausible misclassification, and if it flips, that is the finding.

```
des_all <- svydesign(ids = ~ atom_id, strata = ~ Year, weights = ~ w,
  data = est, nest = TRUE)
obs <- svyby(~ oppose, ~ label, des_all, svymean)
purrr::map(c(0, .05, .10, .20), function(m)
  obs |> dplyr::transmute(label, error_rate = m,
    corrected = round((oppose - m) / (1 - 2 * m), 3))) |>
purrr::list_rbind() |>
tidyr::pivot_wider(names_from = error_rate, values_from = corrected,
  names_prefix = "m=") |>
knitr::kable(caption = "Design-weighted P(oppose) corrected for assumed
symmetric rater error. The m=0 column is the uncorrected estimate.")
```

label	m=0	m=0.05	m=0.1	m=0.2
Topic 13 theme	0.569	0.577	0.587	0.616
Topic 22 theme	0.314	0.293	0.267	0.190
Topic 24 theme	0.086	0.040	-0.018	-0.190

Table 4: Design-weighted $P(\text{oppose})$ corrected for assumed symmetric rater error. The $m=0$ column is the uncorrected estimate.

Data dictionary

Every file this pipeline writes is described in `data-dict.yaml`, generated below from the objects themselves so it cannot drift from what was written. The specification assumes parquet or database tables; these outputs are CSV by project convention, so the document is spec-shaped and human-and-agent readable, but the `data-dict` CLI validator may not run against it unmodified.

```
emit_data_dict(
  path = p_out("data-dict.yaml"),
  name = "public_comment_stance",
  description = str_c("STM topics over a paragraph corpus of public comments,
",
    "LLM topic labels, and design-based stance estimates ",
    "from a stratified two-stage sample."),
```

```

tables = list(
  list(name = "topic_codebook", path = p_out("topic_codebook.csv"),
    data = codebook,
    description = "One row per STM topic: LLM label and description plus
the STM evidence. Analyst-editable; the naming record for coders.",
    vars = list(topic = "STM topic number", label = "Short human-readable
name (analyst-editable)",
      description = "One factual sentence describing the topic",
      prevalence = "Mean theta over all corpus paragraphs",
      n_paras = "Paragraphs whose modal topic is this topic",
      avepp = "Average posterior probability among assigned
paragraphs; below 0.7 indicates fuzzy assignment",
      frex = "Frequent-and-exclusive terms from STM")),
  list(name = "topic_propositions", path = p_out("topic_propositions.csv"),
    data = propositions,
    description = "Stance targets for the selected topics. The proposition
column is the analyst's decision and is quoted verbatim in every stance prompt.",
    vars = list(topic = "STM topic number", label = "Topic label",
      proposition = "The contestable claim stance is measured
toward",
        alt_1 = "Alternative draft", alt_2 = "Alternative draft")),
  list(name = "sample_paragraphs", path = p_out("sample_paragraphs.csv"),
    data = sample_paras,
    description = "The realized probability sample: paragraphs drawn within
selected topics, with design quantities.",
    vars = list(para_uid = "Paragraph id, atom_id-para_id",
      atom_id = "Comment id; the sampling cluster (PSU)",
      Year = "Comment year; the sampling stratum",
      n_tokens = "Whitespace token count", text = "Paragraph
text",
        topic = "Modal STM topic", theta_max = "Modal topic's
theta",
        m_i = "Paragraphs in the comment", N_h = "Comments in the
year stratum",
        n_h = "Comments sampled in the stratum",
        pi1 = "Stage-1 inclusion probability", pi2 = "Stage-2
inclusion probability",
        pi = "Overall inclusion probability", w = "Design weight,
1/pi",
        p_opp = "SIMULATION ONLY: planted P(oppose)",
        true_stance = "SIMULATION ONLY: planted stance")),
  list(name = "stance_labels", path = p_out("stance_labels.csv"),
    data = stance_raw,
    description = "One row per sampled paragraph per rater model.",
    vars = list(para_uid = "Paragraph id", model = "Rater model key",
      stance = "favor, neutral, oppose, or irrelevant")),
  list(name = "gold_to_code", path = p_out("gold_to_code.csv"),
    data = gold_to_code,

```

```

        description = "Blank human-coding template: the coder sees the same
evidence the model saw and no model output.",
        vars = list(para_uid = "Paragraph id", topic = "STM topic number",
                    topic_label = "Topic label", topic_description = "Topic
description",
                    proposition = "The claim to code stance toward", text =
"Paragraph text",
                    human_label = "TO BE FILLED by the coder",
                    coder_notes = "Free-text notes")),
        list(name = "stance_trends", path = p_out("stance_trends.csv"),
            data = trends,
            description = "Design-based fitted P(oppose) by year and topic with 95
percent interval bounds.",
            vars = list(Year = "Calendar year", label = "Topic label",
                        fit = "Fitted P(oppose) from svyglm", lo = "Lower 95%
bound",
                        hi = "Upper 95% bound"))),
        relationships = c(
            "sample_paragraphs.para_uid is a subset of paragraph_theta.para_uid (Script
1)",
            "stance_labels.para_uid joins sample_paragraphs.para_uid, many-to-one across
models",
            "topic_propositions.topic and topic_codebook.topic join on the STM topic
number"),
        glossary = list(
            theta = "STM's posterior topic proportions for a paragraph; rows sum to 1.",
            `modal topic` = "The topic with the highest theta; assignment is an estimate,
not an observation, so it carries classification error.",
            `entropy R2` = "Overall sharpness of modal assignment on a 0-1 scale, as in
latent class analysis.",
            AvePP = "Average posterior probability among a topic's assigned paragraphs;
about 0.7 is the conventional floor.",
            stance = "Position toward a stated proposition (favor, neutral, oppose) or
irrelevant; distinct from sentiment.",
            proposition = "The contestable claim stance is measured toward; analyst-
authored.",
            `design weight` = "1/pi, the number of corpus paragraphs a sampled paragraph
represents.",
            deff = "Design effect; here 1+(b-1)rho from sampling several paragraphs
per comment.))

```

Limitations

Stated plainly, because each bounds a claim this document can support.

1. **Modal assignment carries error.** Topics are model-derived domains, not administrative ones. Entropy R^2 and AvePP report the sharpness, the min_theta floor excludes the fuzziest

paragraphs, and no BCH-style correction is available because STM supplies no classification error matrix over modal assignment.

2. **Labels are description, not validation.** A good label shows the LLM described the cluster STM found; it does not show the cluster is a meaningful theme.
3. **Intervals cover sampling error only.** Rater misclassification is separate and larger, addressed by the sensitivity table, not by the design intervals.
4. **Inter-rater agreement is reliability, not accuracy.** Only the human-coded comparison speaks to accuracy, and only for the classes with adequate support.
5. **Simulated results validate machinery, not models.** Planted-truth recovery shows the estimators work. Synthetic text also states positions more explicitly than real comments, and explicitness is what drives model-human agreement, so those numbers are an upper bound on real-corpus performance, never an estimate of it.